Git & GitHub

Version Control for Our Firmware Projects

Professor: Nerio Andrés Montoya
namontoy@unal.edu.co

2026-S2

Universidad Nacional de Colombia

Agenda

What is Version Control?

What is Git?

Why Git is Powerful

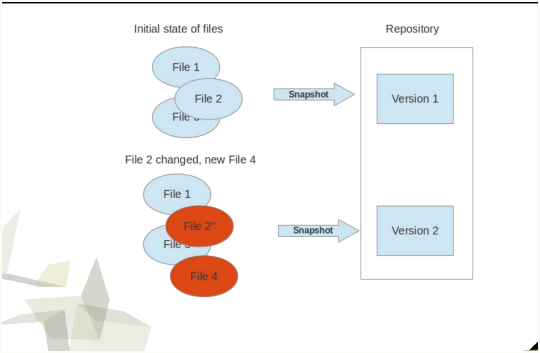
What is GitHub?

Git in Practice

Closing

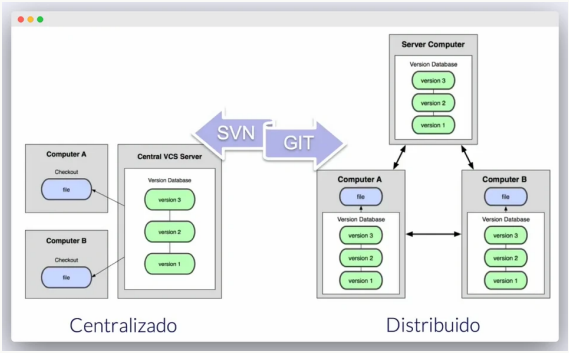
What is Version Control?

The Core Idea: Snapshots



- A version control system stores a *snapshot* of your project every time you save a meaningful checkpoint
- Nothing is ever silently overwritten — every past version stays reachable

Centralized vs. Distributed



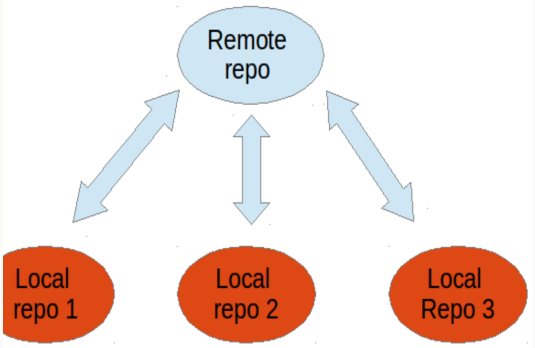
- **Centralized** (e.g. SVN): one server holds the only full history; everyone else checks out a working copy
- **Distributed** (Git): *every* clone holds the *entire* history — your laptop *is* a full backup

What is Git?

What is Git?

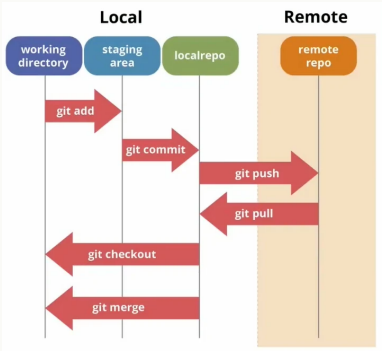
- A **distributed version control system** — created by **Linus Torvalds in 2005** to manage Linux kernel development
- Tracks changes to any set of files over time, by anyone, without needing constant network access
- Today it's the de-facto standard: virtually every serious software (and firmware!) project uses it

Git's Distributed Model



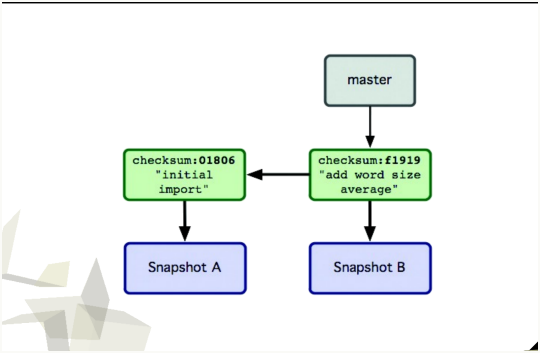
- One remote repository, usually "bare" (no working files, just history)
- One full repository per developer's machine
- Any local repo can rebuild the remote if disaster strikes it

How Git Works, Locally and Remotely



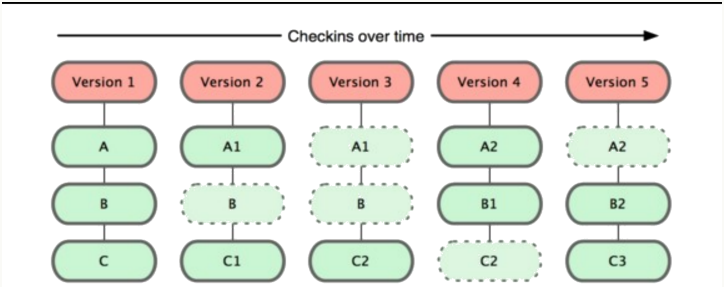
- `add` stages changes; `commit` saves a snapshot locally; `push`/`pull` sync with the remote

Commit History & HEAD



- Every commit points to its parent — history is a chain
- `master` (or `main`) is just a movable label pointing at the latest commit on that line of work

Snapshots Over Time



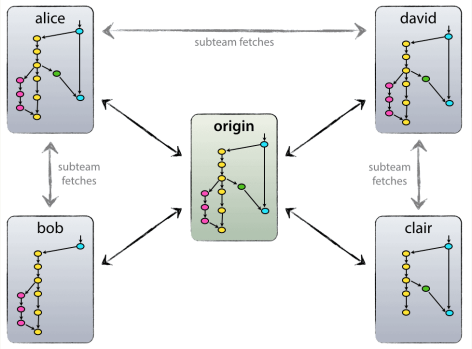
- Only what *changed* is stored per version — unchanged files/blocks are shared between snapshots, not duplicated
- Day to day, you only see the current version; Git quietly keeps the full, ordered history for you

Why Git is Powerful

Why “Distributed” Means “Resilient”

- **No single point of failure** — the remote server can be destroyed and rebuilt from any teammate’s clone
- **Work offline** — commit, branch, inspect history, all without a network connection; sync later
- **Speed** — almost every operation is local, so it’s near-instant compared to talking to a server every time

Peer-to-Peer in Practice

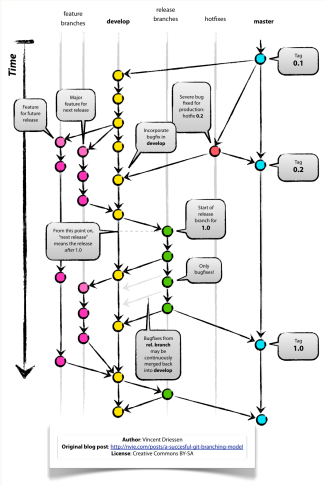


- Teammates can fetch directly from *each other*, not only through `origin`
- Useful for subteams iterating quickly before merging back

[0] Diagram style popularized by the *Pro Git* book (git-scm.com/book)

Branching Without Fear

- Branches in Git are **cheap and instant** — just a pointer, not a full copy
- This makes parallel feature work, experiments, and quick fixes low-risk
- `git-flow` (right) is one popular convention: `develop` for ongoing work, `release` branches to stabilize, `hotfix` for urgent production fixes



What is GitHub?

Git vs. GitHub



- **Git** is the tool — it runs on your machine and manages history. It has *nothing to do* with the internet by itself
- **GitHub** is a company/platform that *hosts* Git repositories in the cloud, plus a web UI and collaboration tools built around them
- Alternatives exist (GitLab, Bitbucket) — same underlying Git, different hosting platform

What GitHub Adds on Top of Git

- **Remote hosting** — a place for `origin` to live, reachable by your whole team
- **Pull Requests** — propose changes, get them reviewed and discussed *before* they merge
- **Issues & Projects** — track bugs, tasks, and progress next to the code itself
- **Actions** — automatically build/test your firmware on every push (CI/CD)
- **Visibility** — stars, forks, your public commit history: a working portfolio for your resume

Why We Use GitHub in This Course

- Submit and version your STM32CubeIDE projects — no more `proyecto_final_v3_definitivo.zip`
- Collaborate with your lab partner without emailing folders back and forth
- I can review your commit history, not just your final submission — it shows how you actually worked
- You leave the course with a public repository you can show in interviews

Git in Practice

The Basic Workflow

```
git clone <url>          # get a copy of a remote repo
git status                # what changed?
git add <file>            # stage changes
git commit -m "message"   # save a snapshot locally
git push                  # send commits to the remote
git pull                  # bring down others' commits
```

Branching Commands

```
git branch                    # list branches
git checkout -b feature/adc   # create + switch to a new branch
git checkout develop          # switch back
git merge feature/adc         # bring the branch's work in
```

- Rule of thumb: one branch per feature or fix, merged back when it's ready

Closing

Key Takeaways

- Git tracks *snapshots*, not just diffs — and every clone holds the entire history
- Distributed means resilient: no single point of failure, and most work happens offline at full speed
- Branches are cheap — use them liberally
- GitHub is not Git — it's where Git repositories live online, plus the tools your team collaborates around
- Starting today: every lab and project goes in a repo